

Smart Store — Loja Autônoma

Eduardo Lopes — Estudo de caso técnico, agosto de 2026

Sistema de varejo autônomo construído de ponta a ponta: do domínio em C# ao firmware que roda no microcontrolador. **MVP funcional, validado em hardware real.**

O que foi construído

Uma loja que opera sem operador de caixa. O cliente entra com QR code, retira produtos, e a compra é montada por leitura RFID. Três camadas integradas: aplicação web, API com banco relacional, e hardware de borda.

Passo a passo do que foi feito

1. Domínio e banco de dados

Modelagem de 8 entidades em .NET 8, com as regras de negócio dentro das próprias entidades — `StoreSession` valida o próprio QR code, expira sozinha e soma itens. Persistência com Entity Framework Core 8 sobre SQL Server: 6 repositórios, 12 migrations, índice único em código de barras e em tag RFID.

2. API REST

ASP.NET Core 8 com 9 controllers, autenticação JWT, login social com Google, recuperação de senha por SMTP e documentação Swagger. Dois endpoints anônimos por decisão de projeto, porque são consumidos por hardware sem sessão de usuário: consulta da sessão aberta e adição de item por tag RFID.

3. Aplicações

- **ClientApp** — Blazor WebAssembly, 15 páginas: catálogo, carrinho ao vivo, abertura da loja, checkout, histórico de compras, perfil
- **AdminApp** — Blazor WebAssembly: produtos, vendas, monitor de prateleira e o campo de vinculação de tag RFID
- **EdgeDesktop** — WPF com CommunityToolkit.Mvvm, aplicação local da loja

4. Inteligência artificial

Integração com Google Gemini em duas frentes: assistente de dúvidas para o cliente e visão computacional que compara quadros da prateleira para identificar o produto retirado, servindo como validação cruzada da leitura RFID.

5. Hardware de borda

ESP32 DevKit V1 com leitor RFID MFRC522 no barramento SPI, display OLED SSD1306 no I2C, sensor de presença HC-SR501 e LED indicador. Montagem em protoboard documentada etapa por etapa.

6. Firmware em C#

O firmware roda sobre **.NET nanoFramework** — C# executando no microcontrolador, não C++. Uma única linguagem do domínio ao dispositivo.

O driver do MFRC522 foi **escrito à mão**, cerca de 200 linhas trabalhando direto com os registradores do chip: comandos REQA e anticolisão do ISO 14443-3, leitura de UID com verificação de BCC. Foi necessário porque a biblioteca oficial do leitor e a de Wi-Fi exigem versões incompatíveis do mesmo assembly, e o nanoFramework resolve dependências por versão exata, sem *binding redirect*.

7. Integração completa

O ciclo que fecha o projeto: tag lida pelo ESP32 → enviada por Wi-Fi à API → produto resolvido no SQL Server → item somado à sessão → estoque decrementado. Tudo funcionando em hardware real.

Decisões técnicas que valem destaque

Entrada por token, nunca por Id. O endpoint de confirmação de entrada recebe o conteúdo lido do QR code e se recusa a aceitar o identificador da sessão. Saber o Id não pode ser suficiente para abrir a porta.

Expiração automática de sessão. Sessão aberta sem checkout por 60 minutos se cancela sozinha. Sem isso, o cliente que fechasse o app ficava permanentemente bloqueado.

Diagnóstico por LED com protocolo próprio. Sem saída de texto confiável no microcontrolador, o LED da placa virou canal de diagnóstico: três piscadas na partida, duas com Wi-Fi conectado, três com sessão encontrada, e grupos distintos para cada falha. Cada arquivo de firmware traz a tabela de sinais documentada.

MVP × produto real: RFID UHF

Este MVP usa RFID 13,56 MHz, que é a tecnologia do leitor RC522. O alcance é de centímetros, então cada produto precisa ser aproximado do leitor na saída. A escolha foi deliberada: hardware acessível, suficiente para validar o fluxo completo de software.

O produto real usaria RFID UHF, na faixa de 860 a 960 MHz. O alcance passa de centímetros para metros, e o leitor identifica dezenas de etiquetas simultaneamente. Na

prática, isso elimina o gesto de passar produto por leitor: o cliente sai da loja com as compras na sacola e o portal de saída lê tudo de uma vez, aprovando o pagamento sem nenhuma ação dele.

É o que caracteriza varejo verdadeiramente sem fricção — e é a diferença entre um protótipo que demonstra o conceito e um sistema que pode operar comercialmente.

A arquitetura de software não muda. O leitor UHF envia tags para o mesmo endpoint, com o mesmo contrato. O que muda é o hardware de borda e o custo por etiqueta. Essa independência entre software e tecnologia de leitura foi um objetivo do desenho, não coincidência.

Próximos passos

- Segredos fora do código, em User Secrets e variáveis de ambiente
 - Entrada autônoma por código de barras, sem depender de aprovação do administrador
 - Fechamento da compra pelo ClientApp
 - Cobertura de testes no domínio
 - Gateway de pagamento
 - Migração para RFID UHF
-

Stack

.NET 8 · ASP.NET Core · Entity Framework Core 8 · SQL Server · Blazor WebAssembly · WPF · JWT · Google OAuth · Google Gemini · .NET nanoFramework · ESP32 · SPI · I2C · MFRC522 · SSD1306